

# Python Programming Fundamentals for Deep Learning

**Instructor:** Samaher Alzahrani

**Date:** 11 May, 2026

---

## Class 2: Python Libraries for Machine Learning

### Topics Covered in This Class:

- Introduction to NumPy
- Linear Algebra with NumPy
- Introduction to Vectorized Operations with NumPy
- Numpy Boolean Array.
- Introduction to Pandas Series
- Introduction to Pandas DataFrames
- Introduction to Plotting with Pandas and Matplotlib.pyplot



---

## Introduction to Numpy

NumPy (Numerical Python) is one of the core packages for numerical computing in Python. Pandas, Matplotlib, Statmodels and many other Scientific libraries rely on NumPy.



## Efficiency in NumPy

One of the primary reasons for NumPy's performance advantage lies in how it represents and processes data.

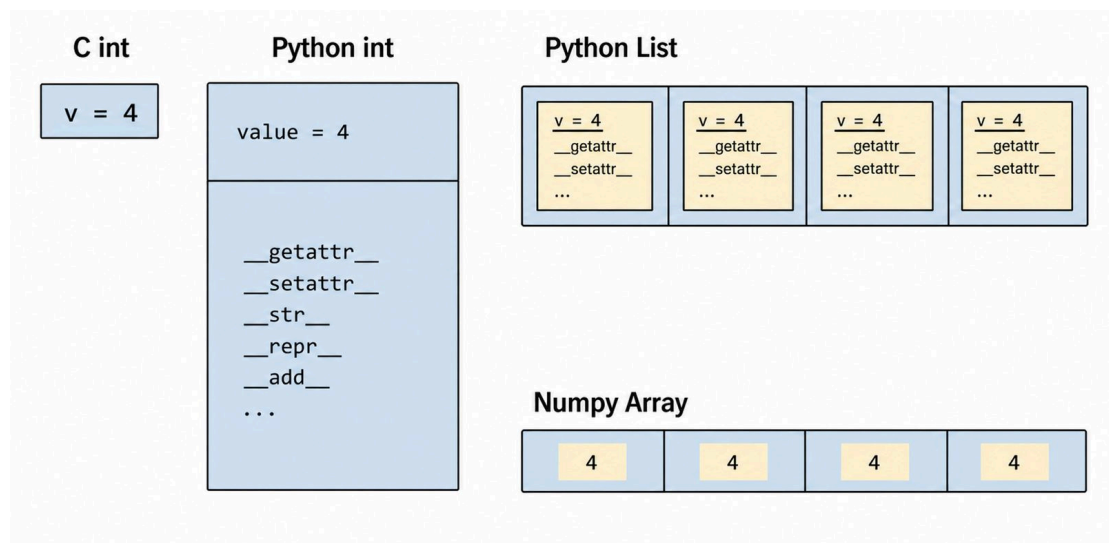
In standard Python, everything is treated as an object, including simple numeric values such as integers. These are often referred to as boxed integers, meaning that each value is wrapped in a full Python object. This object carries additional metadata and functionality, which increases memory usage and computational overhead.

In contrast, NumPy arrays are designed to store data using primitive numeric types (such as fixed-size integers and floating-point numbers). These values are stored in contiguous blocks of memory without the extra overhead associated with Python objects.

As a result:

- Memory usage is significantly reduced, since only the raw data is stored
- Computations are faster, as operations are performed directly on the underlying data
- Vectorized operations can be applied efficiently across entire arrays without explicit Python loops

This design enables NumPy to achieve performance levels close to those of lower-level languages while maintaining the usability of Python.



```
In [1]: x = 4
```

```
In [1]: import numpy as np
```

```
In [2]: np.random.randn(2, 4)
```

```
Out[2]: array([[ 0.42223959,  2.12336061,  0.64892202, -1.13787494],
                [ 0.75173848,  0.31105519, -1.16736934, -0.36774987]])
```

## Basic Numpy Arrays

```
In [4]: np.array([1, 2, 3, 4])
```

```
Out[4]: array([1, 2, 3, 4])
```

```
In [5]: a = np.array([1, 2, 3, 4])
```

```
In [6]: b = np.array([0, .5, 1, 1.5, 2])
```

```
In [7]: a[0], a[1]
```

```
Out[7]: (np.int64(1), np.int64(2))
```

```
In [8]: a[0:]
```

```
Out[8]: array([1, 2, 3, 4])
```

```
In [9]: a[1:3]
```

```
Out[9]: array([2, 3])
```

```
In [10]: a[1:-1]
```

```
Out[10]: array([2, 3])
```

```
In [11]: a[::2]
```

```
Out[11]: array([1, 3])
```

```
In [12]: b[0], b[2], b[-1]
```

```
Out[12]: (np.float64(0.0), np.float64(1.0), np.float64(2.0))
```

```
In [13]: b[[0, 2, -1]]
```

```
Out[13]: array([0., 1., 2.])
```

## Array Types

```
In [14]: a
```

```
Out[14]: array([1, 2, 3, 4])
```

```
In [15]: a.dtype
```

```
Out[15]: dtype('int64')
```

```
In [16]: b
```

```
Out[16]: array([0. , 0.5, 1. , 1.5, 2. ])
```

```
In [17]: b.dtype
```

```
Out[17]: dtype('float64')
```

```
In [18]: c = np.array(['a', 'b', 'c'])
```

```
In [19]: c.dtype # Unicode character Long
```

```
Out[19]: dtype('<U1')
```

## Multidimensional Array

```
In [20]: A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

```
In [21]: A.shape
```

```
Out[21]: (2, 3)
```

```
In [22]: A.ndim
```

```
Out[22]: 2
```

```
In [23]: A.size
```

```
Out[23]: 6
```

```
In [24]: B = np.array([
    [
        [12, 11, 10],
        [9, 8, 7],
    ],
    [
        [6, 5, 4],
        [3, 2, 1]
    ]
])
```

```
In [25]: B
```

```
Out[25]: array([[[12, 11, 10],
                  [ 9,  8,  7]],
                [[ 6,  5,  4],
                  [ 3,  2,  1]]])
```

```
In [26]: B.shape
```

```
Out[26]: (2, 2, 3)
```

```
In [27]: B.ndim
```

```
Out[27]: 3
```

```
In [28]: B.size
```

```
Out[28]: 12
```

# Indexing and Slicing of Matrices

```
In [29]: # Square matrix
A = np.array([
    [1, 2, 3], # 0
    [4, 5, 6], # 1
    [7, 8, 9] # 2
])
```

```
In [30]: A[1]
```

```
Out[30]: array([4, 5, 6])
```

```
In [31]: A[1][0]
```

```
Out[31]: np.int64(4)
```

```
In [32]: A[1, 0]
```

```
Out[32]: np.int64(4)
```

```
In [33]: A[0:2]
```

```
Out[33]: array([[1, 2, 3],
               [4, 5, 6]])
```

```
In [34]: A[:, :2]
```

```
Out[34]: array([[1, 2],
               [4, 5],
               [7, 8]])
```

```
In [35]: A[:, 2]
```

```
Out[35]: array([[1, 2],
               [4, 5]])
```

```
In [36]: A[:, 2:]
```

```
Out[36]: array([[3],
               [6]])
```

```
In [37]: A[1] = np.array([10, 10, 10])
```

```
In [38]: A
```

```
Out[38]: array([[ 1,  2,  3],
               [10, 10, 10],
               [ 7,  8,  9]])
```

```
In [39]: A[2] = 9
```

```
In [40]: A
```

```
Out[40]: array([[ 1,  2,  3],  
                [10, 10, 10],  
                [ 9,  9,  9]])
```

---

# Introduction to Linear Algebra with NumPy

Numpy arrays, when created with the correct set of dimensions, will resemble algebraic structures like vectors or matrices. As expected, `NumPy` will be able to perform the expected primitive operations associated with them. It has a very comprehensive linear algebra sub-system.

```
In [ ]: import numpy as np
```

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

```
In [2]: A = np.array([[1, 2, 3], [4, 5, 6]])  
A
```

```
Out[2]: array([[1, 2, 3],  
               [4, 5, 6]])
```

$$B = \begin{bmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \end{bmatrix}$$

```
In [3]: B = np.full((2, 3), 5)  
B
```

```
Out[3]: array([[5, 5, 5],  
               [5, 5, 5]])
```

$$v = \begin{bmatrix} 6 \\ 7 \\ 8 \end{bmatrix}$$

```
In [4]: v = np.array([6, 7, 8])
```

## Operations with Matrices and Vectors

**Sum of matrices:**

$$A + B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \end{bmatrix} = \begin{bmatrix} 6 & 7 & 8 \\ 9 & 10 & 11 \end{bmatrix}$$

```
In [5]: A + B
```

```
Out[5]: array([[ 6,  7,  8],
               [ 9, 10, 11]])
```

### Subtraction of matrices:

$$A - B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} - \begin{bmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \end{bmatrix} = \begin{bmatrix} -4 & -3 & -2 \\ -1 & 0 & 1 \end{bmatrix}$$

```
In [6]: A - B
```

```
Out[6]: array([[ -4,  -3,  -2],
               [-1,   0,   1]])
```

### Hadamard product (entrywise product):

(Not to be confused with matrix multiplication or dot product):

$$A \circ B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \circ \begin{bmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \end{bmatrix} = \begin{bmatrix} 5 & 10 & 15 \\ 20 & 25 & 30 \end{bmatrix}$$

```
In [7]: A * B # this product is elementwise
```

```
Out[7]: array([[ 5, 10, 15],
               [20, 25, 30]])
```

Division also works "per-entry":

```
In [8]: A / B
```

```
Out[8]: array([[0.2, 0.4, 0.6],
               [0.8, 1. , 1.2]])
```

### Matrix multiplication:

$$A \cdot v = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 7 \\ 8 \end{bmatrix} = \begin{bmatrix} 44 \\ 107 \end{bmatrix}$$

```
In [9]: np.dot(A, v)
```

```
Out[9]: array([ 44, 107])
```

In the latest versions of Python, the `@` operator was introduced for matrix multiplication:

```
In [10]: A @ v
```

```
Out[10]: array([ 44, 107])
```

### Matrix transpose:

```
In [11]: A.T
```

```
Out[11]: array([[1, 4],
               [2, 5],
               [3, 6]])
```

## The identity matrix

An identity matrix is a **square** matrix, of size  $n$ , with ones on the main diagonal and zeros elsewhere:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

So, for example, an identity matrix of size 3 :

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```
In [12]: I = np.identity(3)
I
```

```
Out[12]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

$$C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

```
In [13]: np.arange(1, 10)
```

```
Out[13]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [14]: C = np.arange(1, 10).reshape((3, 3))
C
```

```
Out[14]: array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

Properties of the identity matrix:

$$A_n \cdot I_n = A_n$$

$$I_n \cdot A_n = A_n$$

```
In [15]: I @ C
```

```
Out[15]: array([[1., 2., 3.],
               [4., 5., 6.],
               [7., 8., 9.]])
```

```
In [16]: C @ I
```

```
Out[16]: array([[1., 2., 3.],
               [4., 5., 6.],
               [7., 8., 9.]])
```

## Inverse of a matrix:

The formula to find the inverse of a `2x2` matrix is:

$$A^{-1} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}^{-1} = \frac{1}{|A|} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

`|A|` is the determinant of a matrix, which can be computed with `np.linalg.det`. So, as an example, given the following matrix:

$$D = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```
In [17]: D = np.array([
           [1, 2],
           [3, 4]
         ])
D
```

```
Out[17]: array([[1, 2],
               [3, 4]])
```

The determinant can be calculated using:

```
In [18]: np.linalg.det(D) # check determinate
```

```
Out[18]: -2.0000000000000004
```

And the inverse with `np.linalg.inv`:

```
In [19]: np.linalg.inv(D) # find the inverse
```

```
Out[19]: array([[-2. ,  1. ],
               [ 1.5, -0.5]])
```

The main characteristic of the inverse matrix is that, it acts as a reciprocal of the matrix:

```
In [22]: D @ np.linalg.inv(D) # @ is multiplication symbol
```

```
Out[22]: array([[1.0000000e+00, 0.0000000e+00],
               [8.8817842e-16, 1.0000000e+00]])
```

```
In [ ]: np.round(D @ np.linalg.inv(D))
```

```
Out[ ]: array([[1., 0.],
               [0., 1.]])
```

## Useful Numpy functions

We'll now take the chance to see in more detail the functions we've been using, the most common and useful ones in the `numpy` world:

**arange** return evenly spaced values within a given interval.

- `range()` → used in loops and iteration.
- `np.arange()` → used for numerical operations and arrays.

```
In [ ]: range(5) # range object (not a list)
```

```
Out[ ]: range(0, 5)
```

```
In [1]: list(range(5))
```

```
Out[1]: [0, 1, 2, 3, 4]
```

```
In [25]: np.arange(10)
```

```
Out[25]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [26]: np.arange(5, 10)
```

```
Out[26]: array([5, 6, 7, 8, 9])
```

```
In [ ]: np.arange(0, 1, .1)
```

```
Out[ ]: array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
```

### **linspace**

Return evenly spaced numbers over a specified interval.

```
In [ ]: np.linspace(0, 1, 5)
```

```
Out[ ]: array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

```
In [29]: np.linspace(0, 1, 20)
```

```
Out[29]: array([0. , 0.05263158, 0.10526316, 0.15789474, 0.21052632,
               0.26315789, 0.31578947, 0.36842105, 0.42105263, 0.47368421,
               0.52631579, 0.57894737, 0.63157895, 0.68421053, 0.73684211,
               0.78947368, 0.84210526, 0.89473684, 0.94736842, 1.  ])
```

```
In [30]: np.linspace(0, 1, 20, False) # does approximate the point, not efficient in math
```

```
Out[30]: array([0. , 0.05, 0.1 , 0.15, 0.2 , 0.25, 0.3 , 0.35, 0.4 , 0.45, 0.5 ,
               0.55, 0.6 , 0.65, 0.7 , 0.75, 0.8 , 0.85, 0.9 , 0.95])
```

### **zeros , ones , empty**

```
In [31]: np.zeros(5)
```

```
Out[31]: array([0., 0., 0., 0., 0.])
```

```
In [32]: np.zeros((3, 3))
```

```
Out[32]: array([[0., 0., 0.],
               [0., 0., 0.],
               [0., 0., 0.]])
```

```
In [33]: np.zeros((3, 3), dtype=np.int)
```

```
Out[33]: array([[0, 0, 0],
               [0, 0, 0],
               [0, 0, 0]])
```

```
In [34]: np.ones(5)
```

```
Out[34]: array([1., 1., 1., 1., 1.])
```

```
In [35]: np.ones((3, 3))
```

```
Out[35]: array([[1., 1., 1.],
               [1., 1., 1.],
               [1., 1., 1.]])
```

### **Full filled**

```
In [38]: np.full((2,2), 7)
```

```
Out[38]: array([[7, 7],
               [7, 7]])
```

```
In [39]: np.full((3, 3), 1)
```

```
Out[39]: array([[1, 1, 1],
               [1, 1, 1],
               [1, 1, 1]])
```

### **Random filled**

```
In [40]: np.random.random((2,2))
```

```
Out[40]: array([[0.95994826, 0.11944492],
               [0.67259477, 0.26004165]])
```

### **Identity and eye**

```
In [41]: np.identity(3)
```

```
Out[41]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

```
In [42]: np.eye(3, 3)
```

```
Out[42]: array([[1., 0., 0.],  
               [0., 1., 0.],  
               [0., 0., 1.]])
```

```
In [43]: np.eye(8, 4)
```

```
Out[43]: array([[1., 0., 0., 0.],  
               [0., 1., 0., 0.],  
               [0., 0., 1., 0.],  
               [0., 0., 0., 1.],  
               [0., 0., 0., 0.],  
               [0., 0., 0., 0.],  
               [0., 0., 0., 0.],  
               [0., 0., 0., 0.]])
```

---

# Vectorized Operations with NumPy

"Vectorization" is an important concept in numeric computing. It refers to the process of operating in entire arrays, in a concise and efficient way. Let's take a look with a couple of examples.

Numpy also refers to *vectorized operations* as "**array broadcasting**".

```
In [34]: import numpy as np
```

```
In [35]: a = np.arange(0, 5)
a
```

```
Out[35]: array([0, 1, 2, 3, 4])
```

```
In [36]: b = np.arange(5, 10)
b
```

```
Out[36]: array([5, 6, 7, 8, 9])
```

A vectorized operation is any algebraic binary operation you perform on them:

```
In [37]: a + b # elementwise operation
```

```
Out[37]: array([ 5,  7,  9, 11, 13])
```

In this case, the operation was performed "element-wise", ie: "vectorized". Let's compare it with a regular for-loop operation:

```
In [38]: res = []
for i in range(len(a)):
    res.append(a[i] + b[i])
np.array(res)
```

```
Out[38]: array([ 5,  7,  9, 11, 13])
```

It's much easier to express the operation as `a + b`, instead of explicitly defining the for loop to perform the element-wise sum. This is the difference between *declarative* and *imperative* programming.

Finally, compare this to the usage of regular Python lists:

```
In [39]: l1 = list(range(5))
l1
```

```
Out[39]: [0, 1, 2, 3, 4]
```

```
In [40]: l2 = list(range(5, 10))
l2
```

```
Out[40]: [5, 6, 7, 8, 9]
```

```
In [41]: 11 + 12 # sum of lists is not elementwise summing it just adding new elements to
```

```
Out[41]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Python lists don't operate "algebraically", the `+` operation is just concatenation.

Generally, array vectorization will always work with arrays of the same size and shape, but it can also work for arrays of different shapes. For example:

```
In [42]: A = np.array([
          [0, 0, 0],
          [1, 1, 1],
          [2, 2, 2],
          [3, 3, 3]
        ])
A
```

```
Out[42]: array([[0, 0, 0],
               [1, 1, 1],
               [2, 2, 2],
               [3, 3, 3]])
```

```
In [43]: A.shape
```

```
Out[43]: (4, 3)
```

```
In [44]: b = np.array([[1, 2, 3]])
b
```

```
Out[44]: array([[1, 2, 3]])
```

```
In [45]: b.shape
```

```
Out[45]: (1, 3)
```

Even though the shapes are different, we can still perform a vectorized operation between `A` and `b` :

```
In [46]: A + b # it did elementwise summing for each row in A
```

```
Out[46]: array([[1, 2, 3],
               [2, 3, 4],
               [3, 4, 5],
               [4, 5, 6]])
```

This is possible since:

**The Broadcasting Rule:** Two arrays are compatible for broadcasting if for each *trailing dimension* (i.e., starting from the end) the axis lengths match or if either of the lengths is 1. Broadcasting is then performed over the missing or length 1 dimensions.

Any operation between arrays not following *The Broadcasting Rule*, will fail with a "broadcast" related error. For example:

```
In [47]: A = np.arange(6).reshape(2, 3)
A
```

```
Out[47]: array([[0, 1, 2],
               [3, 4, 5]])
```

```
In [48]: b = np.array([5, 5])
b
```

```
Out[48]: array([5, 5])
```

```
In [49]: A + b # we can't perform, can't broadcast because the shapes are not compatible
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[49], line 1
----> 1 A + b # we can't perform, can't broadcast because the shapes are not compatible

ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

```
In [ ]: a = np.arange(0, 6)
a
```

```
Out[ ]: array([0, 1, 2, 3, 4, 5])
```

```
In [ ]: a + 10 # perform over the whole array, this notion fails for lists.
```

```
Out[ ]: array([10, 11, 12, 13, 14, 15])
```

As you can see, the `+ 10` operation was "broadcasted" (or "distributed") among all the elements of the array. This is what we usually do with a list comprehension in pure Python:

```
In [ ]: [x + 10 for x in a]
```

```
Out[ ]: [10, 11, 12, 13, 14, 15]
```

But this "broadcasting" behavior is default for Numpy. Here are a few other examples:

```
In [ ]: a - 5
```

```
Out[ ]: array([-5, -4, -3, -2, -1,  0])
```

```
In [ ]: a * 3
```

```
Out[ ]: array([ 0,  3,  6,  9, 12, 15])
```

## Numpy Boolean Array

This kind of arrays help us to perform some sort of filtration of data that we would use to machine learning.

```
In [ ]: a > 2 # it did check element by element, it doesn't work with the lists
```

```
Out[ ]: array([False, False, False,  True,  True,  True])
```

```
In [ ]: a == 0
```

```
Out[ ]: array([ True, False, False, False, False, False])
```

```
In [ ]: c = np.arange(0,20,3)
c
```

```
Out[ ]: array([ 0,  3,  6,  9, 12, 15, 18])
```

```
In [ ]: c % 2 == 0 # Modular Arithmetic
```

```
Out[ ]: array([ True, False,  True, False,  True, False,  True])
```

A common practice is to use the function `np.sum` to count how many elements in the array satisfy a given condition. For example, from our array `a`, how many elements are greater than 2 (`> 2`)? The answer is `3` (elements `3`, `4` and `5`).

```
In [ ]: a > 2 # 0 is False and 1 is True, so we can perform sum function.
```

```
Out[ ]: array([False, False, False,  True,  True,  True])
```

```
In [ ]: np.sum(a > 2)
```

```
Out[ ]: np.int64(2)
```

Finally, it's important to note that vectorized operations (algebraic and boolean) with one dimensional arrays will also work for multidimensional arrays:

```
In [ ]: A = np.arange(1, 10).reshape(3, 3)
A
```

```
Out[ ]: array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

```
In [ ]: A + 100
```

```
Out[ ]: array([[101, 102, 103],
               [104, 105, 106],
               [107, 108, 109]])
```

```
In [ ]: A % 2
```

```
Out[ ]: array([[1, 0, 1],
               [0, 1, 0],
               [1, 0, 1]])
```

```
In [ ]: A % 2 == 0
```

```
Out[ ]: array([[False,  True, False],
               [ True, False,  True],
               [False,  True, False]])
```

## Logical Operation

You're probably already familiar with python's logical operators ( `and` , `or` and `not` ). We'll see now `numpy` 's counterparts. From now on, this table might be useful:

| Python           | Numpy              |
|------------------|--------------------|
| <code>and</code> | <code>&amp;</code> |
| <code>or</code>  | <code> </code>     |
| <code>not</code> | <code>~</code>     |

```
In [53]: a
```

```
Out[53]: array([0, 1, 2, 3, 4])
```

```
In [ ]: (a >= 2) & (a < 5)
```

```
Out[ ]: array([False, False,  True,  True,  True])
```

```
In [ ]: ~(a <= 2)
```

```
Out[ ]: array([False, False, False,  True,  True])
```

## Logical Operators in Filtering

```
In [ ]: a[(a >= 2) & (a < 5)]
```

```
Out[ ]: array([2, 3, 4])
```

```
In [ ]: a[a >= 3] = 99
```

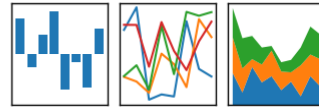
```
In [ ]: a
```

```
Out[ ]: array([ 0,  1,  2, 99, 99])
```

# Introduction to Pandas Series

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



A Series is a one-dimensional array-like object containing a *typed* sequence of values and an associated array of data labels, called its *index*.

```
In [2]: import numpy as np
import pandas as pd
```

```
In [3]: series = pd.Series([1, 2, 3, 4, 5, 6])
series # default numpy datatype is int64
```

```
Out[3]: 0    1
        1    2
        2    3
        3    4
        4    5
        5    6
dtype: int64
```

Series have an associated type:

```
In [4]: # Show first values of our Series
series.head() # by default give first 5
```

```
Out[4]: 0    1
        1    2
        2    3
        3    4
        4    5
dtype: int64
```

```
In [5]: series.dtype
```

```
Out[5]: dtype('int64')
```

```
In [6]: series = pd.Series([1, 2, 3, 4, 5], dtype=float) #coming from numpy
series
```

```
Out[6]: 0    1.0
        1    2.0
        2    3.0
        3    4.0
        4    5.0
dtype: float64
```

```
In [7]: series.dtype
```

```
Out[7]: dtype('float64')
```

```
In [8]: series = pd.Series(['a', 'b', 'c', 'd', 'e'])
series
```

```
Out[8]: 0    a
        1    b
        2    c
        3    d
        4    e
        dtype: object
```

```
In [9]: # Using a ndarraynp.array([2, 4, 6, 8, 10])
array = np.array([2, 4, 6, 8, 10])
series = pd.Series(array) # we can pass array
series
```

```
Out[9]: 0     2
        1     4
        2     6
        3     8
        4    10
        dtype: int64
```

```
In [10]: # With predefined index
series = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e']) ## change t
series
```

```
Out[10]: a     1
         b     2
         c     3
         d     4
         e     5
         dtype: int64
```

```
In [11]: # Using a dictionary (index will be defined using keys)
series = pd.Series({'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}, dtype=np.float64)
series
```

```
Out[11]: a     1.0
         b     2.0
         c     3.0
         d     4.0
         e     5.0
         dtype: float64
```

```
In [12]: # Values of a series
series.values
```

```
Out[12]: array([1., 2., 3., 4., 5.])
```

```
In [13]: type(series.values)
```

```
Out[13]: numpy.ndarray
```

```
In [14]: # Index of a series
series.index
```

```
Out[14]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

```
In [15]: # Shape of the Series  
series.shape
```

```
Out[15]: (5,)
```

```
In [16]: # Number of Series elements  
series.size
```

```
Out[16]: 5
```

Someone might not know we're representing population in millions of inhabitants. Series can have a `name`, to better document the purpose of the Series:

```
In [17]: series.reset_index(drop=True)
```

```
Out[17]: 0    1.0  
        1    2.0  
        2    3.0  
        3    4.0  
        4    5.0  
dtype: float64
```

Note that `reset_index()` will return a new `Series`, so if we want to keep it we need to assign it to a variable, or use `inplace=True` parameter to modify the original `Series`.

```
In [19]: series
```

```
Out[19]: a    1.0  
        b    2.0  
        c    3.0  
        d    4.0  
        e    5.0  
dtype: float64
```

```
In [20]: series.reset_index(drop=True, inplace=True)
```

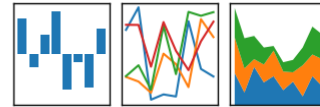
```
In [21]: series
```

```
Out[21]: 0    1.0  
        1    2.0  
        2    3.0  
        3    4.0  
        4    5.0  
dtype: float64
```

# Introduction to Pandas DataFrames

# pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



Probably the most important data structure of pandas is the `DataFrame`. It's a tabular structure tightly integrated with `Series`.

A DataFrame is a tabular structure with the following properties:

- It's composed by a ordered series of rows and a ordered series of columns.
- It also uses an index to reference individual rows.
- Each column could have a different NumPy-related type.
- It could be seen as a collection of multiple of Series, all sharing the same index.
- Can be "sliced" horizontally (per row) or vertically (per column).

```
In [25]: import numpy as np
import pandas as pd
```

## DataFrames creation

The `DataFrame` constructor accepts the following parameters:

- **data:** (required) has all the data we want to store on the DataFrame and could be a Series dictionary, a sequences dictionary, a bidimensional ndarray, a Series or another DataFrame.
- **index:** (optional), has all the labels we want to assign to the rows of our DataFrame and could be a Python sequence or an unidimensional ndarray. Default value: `np.arange(0, len(rows))`.
- **columns:** (optional), has all the labels we want to assign to the columns of our DataFrame and could be a Python sequence or an unidimensional ndarray. Default value: `np.arange(0, len(columns))`.
- **dtype:** (optional) any NumPy data type to be assigned on columns.

```
In [26]: # Using a dictionary of sequences
         dataframe = pd.DataFrame({'col1': [1, 2, 3],
                                   'col2': ['one', 'two', 'three'],
                                   'col3': [1.0, 2.0, 3.0]})

         dataframe
```

Out[26]:

|   | col1 | col2  | col3 |
|---|------|-------|------|
| 0 | 1    | one   | 1.0  |
| 1 | 2    | two   | 2.0  |
| 2 | 3    | three | 3.0  |

```
In [27]: # Using a dictionary of Series
dataframe = pd.DataFrame({'var1': pd.Series([1, 2, 3], dtype=float),
                          'var2': pd.Series(['a', 'b'])})

dataframe
```

Out[27]:

|   | var1 | var2 |
|---|------|------|
| 0 | 1.0  | a    |
| 1 | 2.0  | b    |
| 2 | 3.0  | NaN  |

```
In [28]: # Using a ndarray with indexes to rows and columns
dataframe = pd.DataFrame(np.arange(16).reshape(4, 4),
                          index=['r1', 'r2', 'r3', 'r4'],
                          columns=['c1', 'c2', 'c3', 'c4'])

dataframe
```

Out[28]:

|    | c1 | c2 | c3 | c4 |
|----|----|----|----|----|
| r1 | 0  | 1  | 2  | 3  |
| r2 | 4  | 5  | 6  | 7  |
| r3 | 8  | 9  | 10 | 11 |
| r4 | 12 | 13 | 14 | 15 |

```
In [29]: # Using a ndarray with indexes to rows and columns, with fixed type
dataframe = pd.DataFrame(np.arange(16).reshape(4,4), dtype=np.int32)

dataframe
```

Out[29]:

|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 0  | 1  | 2  | 3  |
| 1 | 4  | 5  | 6  | 7  |
| 2 | 8  | 9  | 10 | 11 |
| 3 | 12 | 13 | 14 | 15 |

```
In [30]: dataframe.dtypes
```

```
Out[30]: 0    int32
         1    int32
         2    int32
         3    int32
         dtype: object
```

---

## DataFrame elements

`DataFrame`s expose some useful attributes:

```
In [31]: dataframe = pd.DataFrame({'col1': pd.Series([1, 2, 3], dtype=np.float64),
                                   'col2': pd.Series(['a', 'b'])})
```

dataframe

```
Out[31]:
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1.0  | a    |
| 1 | 2.0  | b    |
| 2 | 3.0  | NaN  |

```
In [32]: # Show first rows of our DataFrame
dataframe.head()
```

```
Out[32]:
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1.0  | a    |
| 1 | 2.0  | b    |
| 2 | 3.0  | NaN  |

```
In [33]: # Type of our DataFrame columns
dataframe.dtypes
```

```
Out[33]: col1    float64
         col2    object
         dtype: object
```

```
In [34]: dataframe
```

```
Out[34]:
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1.0  | a    |
| 1 | 2.0  | b    |
| 2 | 3.0  | NaN  |

```
In [35]: # Values of a DataFrame
dataframe.values
```

```
Out[35]: array([[1.0, 'a'],
               [2.0, 'b'],
               [3.0, nan]], dtype=object)
```

```
In [36]: dataframe.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   col1     3 non-null        float64
1   col2     2 non-null        object
dtypes: float64(1), object(1)
memory usage: 180.0+ bytes
```

```
In [37]: type(dataframe.values)
```

```
Out[37]: numpy.ndarray
```

```
In [38]: # Using a ndarray with indexes to rows and columns
dataframe = pd.DataFrame(np.arange(16).reshape(4, 4),
                          index=['r1', 'r2', 'r3', 'r4'],
                          columns=['c1', 'c2', 'c3', 'c4'])

dataframe
```

```
Out[38]:
```

|    | c1 | c2 | c3 | c4 |
|----|----|----|----|----|
| r1 | 0  | 1  | 2  | 3  |
| r2 | 4  | 5  | 6  | 7  |
| r3 | 8  | 9  | 10 | 11 |
| r4 | 12 | 13 | 14 | 15 |

```
In [39]: # Index of a DataFrame
dataframe.index
```

```
Out[39]: Index(['r1', 'r2', 'r3', 'r4'], dtype='object')
```

```
In [40]: # Columns of a DataFrame
dataframe.columns
```

```
Out[40]: Index(['c1', 'c2', 'c3', 'c4'], dtype='object')
```

```
In [41]: # Dimension of the DataFrame
dataframe.ndim # two dimensions for table (vertical, horizontal)
```

```
Out[41]: 2
```

```
In [42]: # Shape of the DataFrame
dataframe.shape
```

```
Out[42]: (4, 4)
```

```
In [43]: # Number of DataFrame elements
dataframe.size
```

Out[43]: 16

*Indexes are immutable, so we can't change individual values independently.  
However, we can change a complete index with a new index.*

```
In [44]: dataframe = pd.DataFrame({'var1': pd.Series([1, 2, 3], dtype=np.float64),
                                   'var2': pd.Series(['a', 'b'])})

dataframe
```

```
Out[44]:
```

|   | var1 | var2 |
|---|------|------|
| 0 | 1.0  | a    |
| 1 | 2.0  | b    |
| 2 | 3.0  | NaN  |

```
In [45]: # Modifying a row index will give us an error
dataframe.index[0] = 4
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[45], line 2
      1 # Modifying a row index will give us an error
----> 2 dataframe.index[0] = 4

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\indexes\base.p
y:5371, in Index.__setitem__(self, key, value)
    5369 @final
    5370 def __setitem__(self, key, value) -> None:
-> 5371     raise TypeError("Index does not support mutable operations")

TypeError: Index does not support mutable operations
```

```
In [ ]: # Modifying a column index will give us an error
dataframe.columns[0] = 4
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-22-e244ac47fdd7> in <module>
      1 # Modifying a column index will give us an error
----> 2 dataframe.columns[0] = 4

~\.virtualenvs\jupyter-ext\lib\python3.6\site-packages\pandas\core\indexes\base.p
y in __setitem__(self, key, value)
    4258
    4259     def __setitem__(self, key, value):
-> 4260         raise TypeError("Index does not support mutable operations")
    4261
    4262     def __getitem__(self, key):

TypeError: Index does not support mutable operations
```

```
In [ ]: # This will work
dataframe.index = ['r1', 'r2', 'r3']
dataframe
```

```
Out[ ]:      var1  var2
r1    1.0    a
r2    2.0    b
r3    3.0  NaN
```

```
In [ ]: # This will work
dataframe.columns = ['c1', 'c2']
dataframe
```

```
Out[ ]:      c1  c2
r1    1.0  a
r2    2.0  b
r3    3.0  NaN
```

---

## Note:

In this class, we introduced only the fundamentals of Pandas to build Series and DataFrames, due to time limitations, many important Pandas functionalities and techniques were not covered.

## These include:

- Data cleaning and preprocessing
- Handling missing values
- Detecting and correcting invalid values
- Converting and managing incorrect data types
- Removing duplicate data
- Filtering data using conditional statements
- Boolean indexing and boolean filtering
- String manipulation and text processing
- Date and time handling
- Merging and combining tables
- Grouping and aggregation techniques
- Inner joins and outer joins
- Creating and analyzing pivot tables

Additional resources for data manipulation:

- [Python for Data Analysis](#)
- [Python for Data Science the Ultimate Beginners Guide](#)

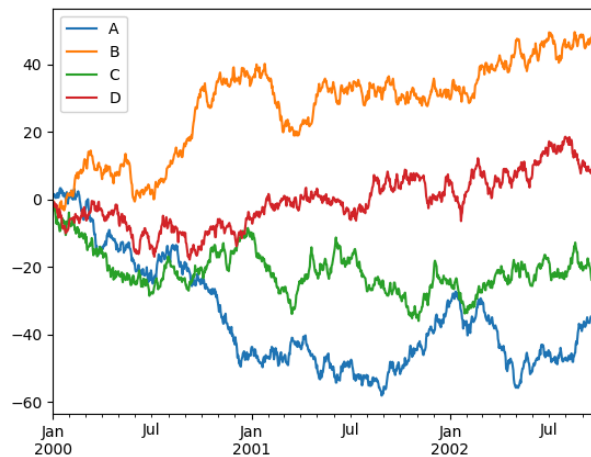
---

# Plotting with Pandas

The pandas library has become popular not just for enabling powerful data analysis, but also for its handy pre-canned plotting methods.

Interestingly though, pandas plotting methods are really just convenient wrappers around existing matplotlib calls.

That is, the `plot()` method on pandas' Series and DataFrame is a wrapper around `plt.plot()`.



```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Pandas can easily read data stored in different file formats like CSV, JSON, XML or even Excel as we saw on the previous lecture.

Let's read some data and plot some basic figures.

```
In [2]: # Load data
df = pd.read_csv('nba_data.csv')

# show first rows
df.head()
```

Out[2]:

|   | Rk | Player            | Position | Age | Mp   | Fg   | Fga  | Fg%   | 3P  | 3Pa | ... | Team | Gp | Mpg  |
|---|----|-------------------|----------|-----|------|------|------|-------|-----|-----|-----|------|----|------|
| 0 | 1  | Russell Westbrook | PG       | 28  | 34.6 | 10.2 | 24.0 | 0.425 | 2.5 | 7.2 | ... | OKC  | 81 | 34.6 |
| 1 | 2  | James Harden      | PG       | 27  | 36.4 | 8.3  | 18.9 | 0.440 | 3.2 | 9.3 | ... | HOU  | 81 | 36.4 |
| 2 | 3  | Isaiah Thomas     | PG       | 27  | 33.8 | 9.0  | 19.4 | 0.463 | 3.2 | 8.5 | ... | BOS  | 76 | 33.8 |
| 3 | 4  | Anthony Davis     | C        | 23  | 36.1 | 10.3 | 20.3 | 0.505 | 0.5 | 1.8 | ... | NO   | 75 | 36.1 |
| 4 | 5  | DeMar DeRozan     | SG       | 27  | 35.4 | 9.7  | 20.9 | 0.467 | 0.4 | 1.7 | ... | TOR  | 74 | 35.4 |

5 rows × 37 columns

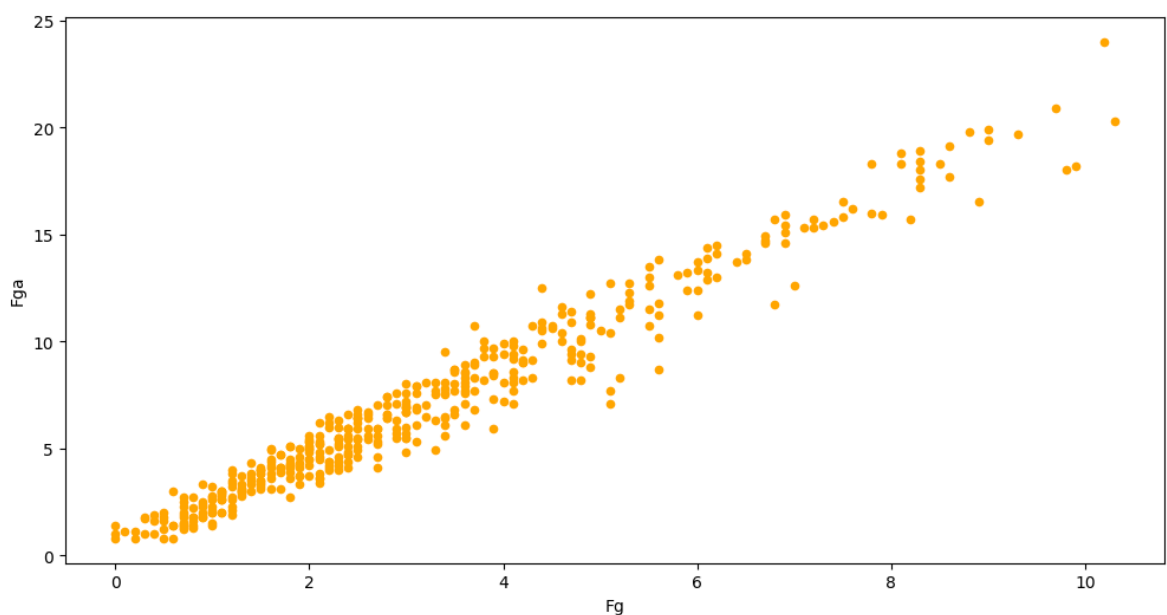


## Scatter plot

We'll use `Fg` and `Fga` to draw a **scatter plot** and see the relation between both columns.

```
In [3]: df.plot(kind='scatter',
               x= 'Fg',
               y= 'Fga',
               color='orange',
               figsize=(12, 6))
```

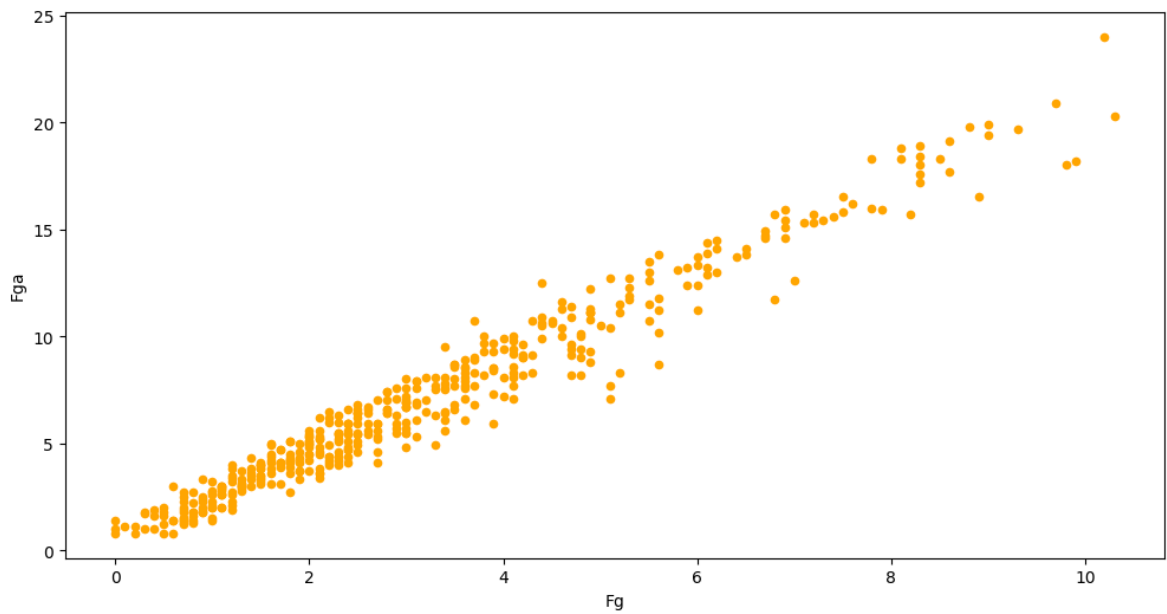
Out[3]: <Axes: xlabel='Fg', ylabel='Fga'>



```
In [4]: df.plot.scatter(x='Fg',
                        y='Fga',
```

```
color='orange',  
figsize=(12, 6))
```

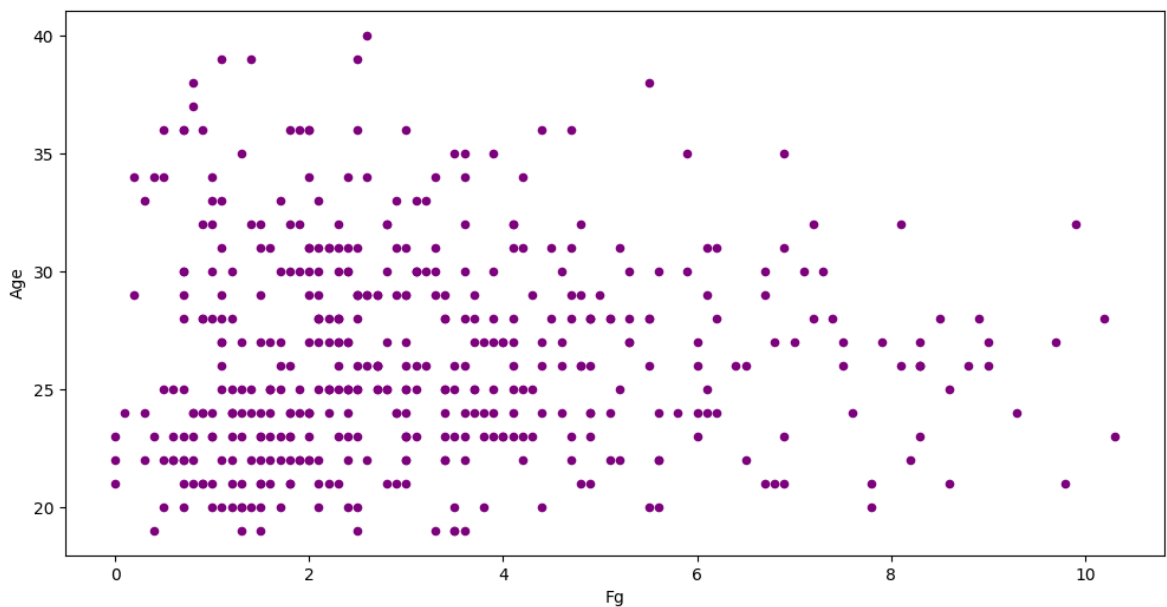
Out[4]: <Axes: xlabel='Fg', ylabel='Fga'>



Now let's see relation between `Fg` and `Age` :

```
In [5]: df.plot(kind='scatter',  
              x='Fg',  
              y='Age',  
              color='purple',  
              figsize=(12, 6))
```

Out[5]: <Axes: xlabel='Fg', ylabel='Age'>



## Bar plot

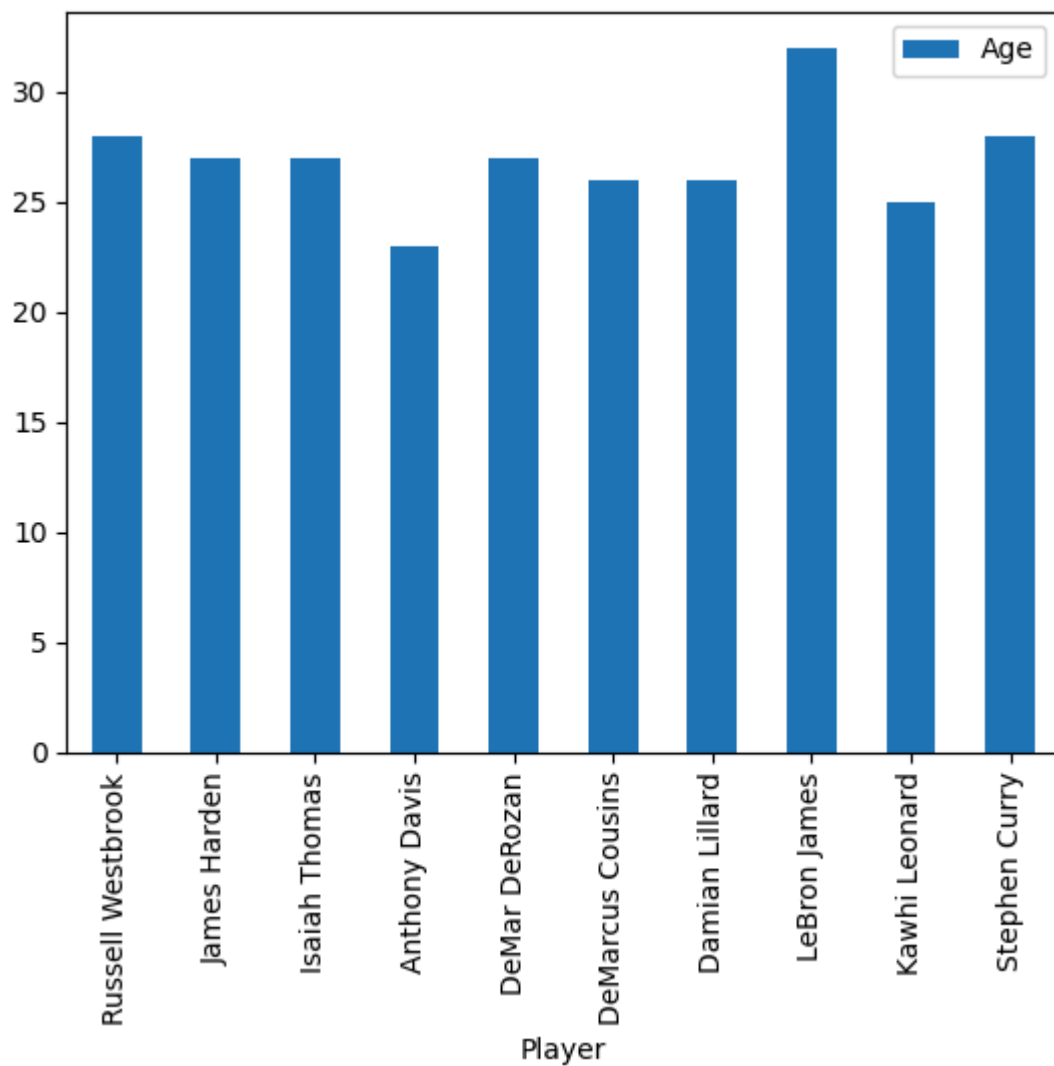
We can also make some nice **bar plots**. Let's see the age of the first 10 players:

```
In [6]: df.head(10)['Age']
```

```
Out[6]: 0    28
        1    27
        2    27
        3    23
        4    27
        5    26
        6    26
        7    32
        8    25
        9    28
        Name: Age, dtype: int64
```

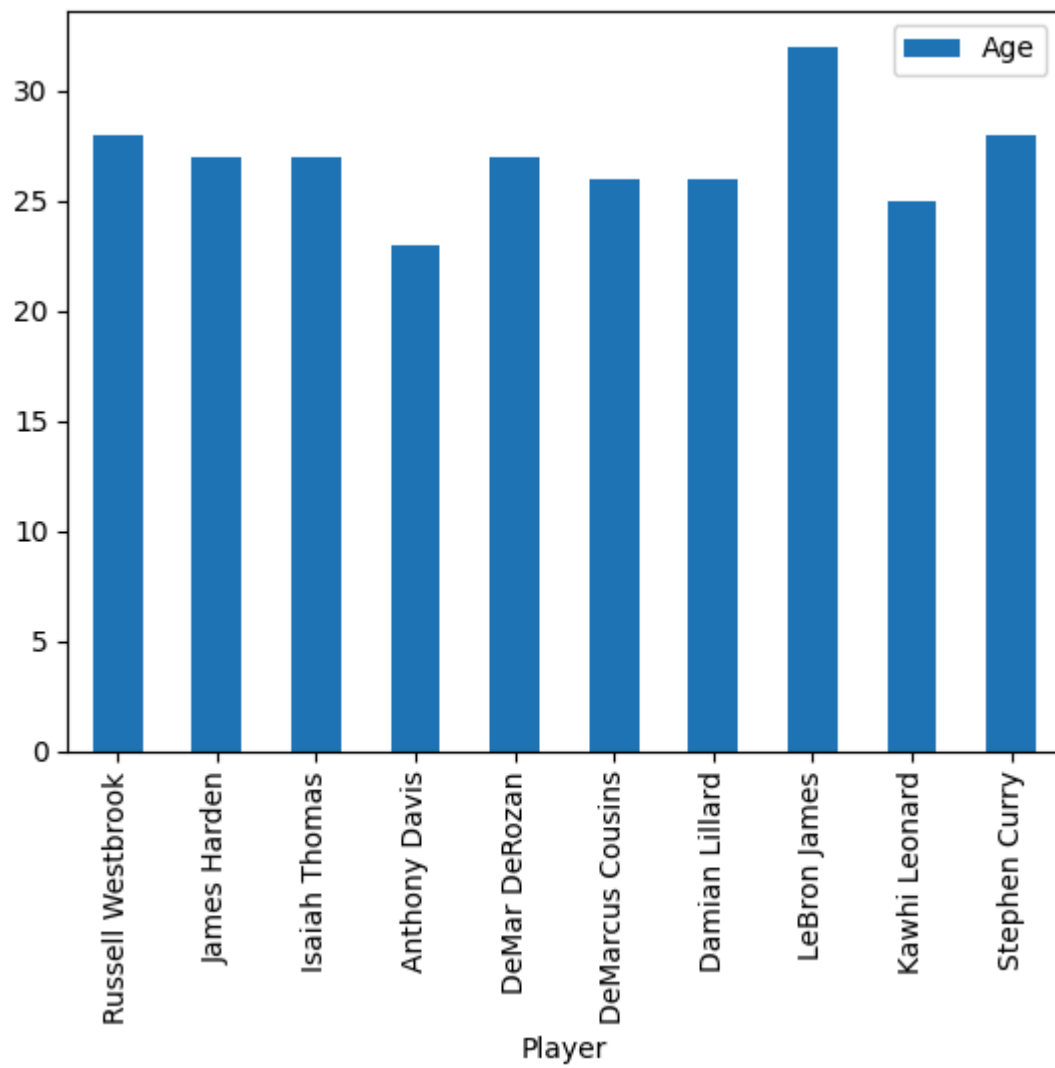
```
In [7]: df.head(10).plot(kind='bar',
                        x='Player',
                        y='Age')
```

```
Out[7]: <Axes: xlabel='Player'>
```



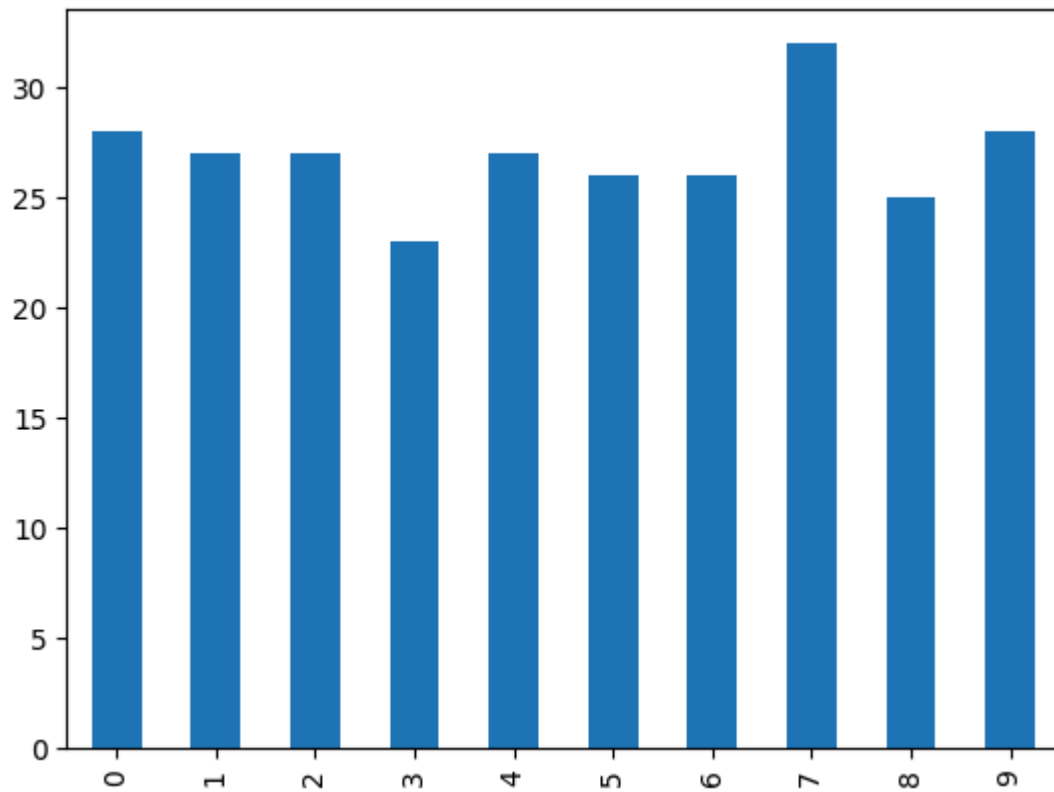
```
In [8]: df.head(10).plot.bar(x='Player',
                        y='Age')
```

```
Out[8]: <Axes: xlabel='Player'>
```



```
In [9]: df.head(10)['Age'].plot(kind='bar')
```

```
Out[9]: <Axes: >
```



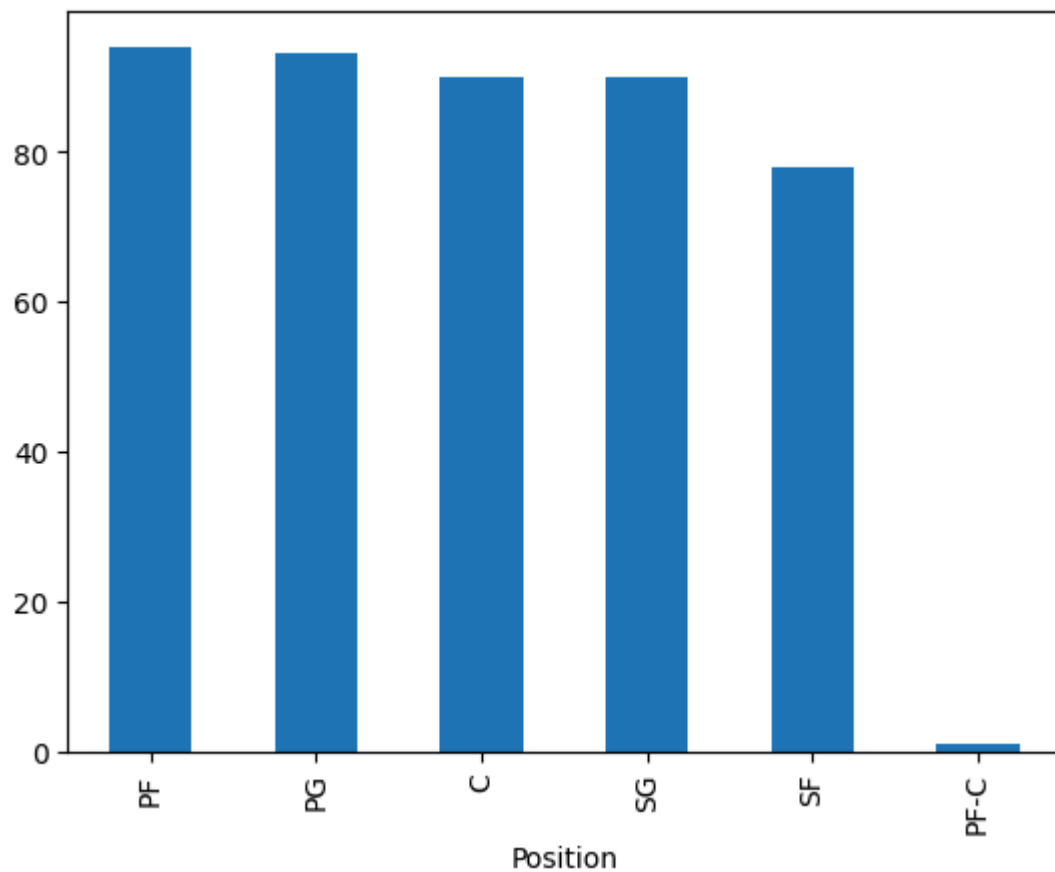
Also is possible with columns without numeric values:

```
In [10]: df['Position'].value_counts()
```

```
Out[10]: Position
PF      94
PG      93
C       90
SG      90
SF      78
PF-C     1
Name: count, dtype: int64
```

```
In [11]: df['Position'].value_counts().plot(kind='bar')
```

```
Out[11]: <Axes: xlabel='Position'>
```

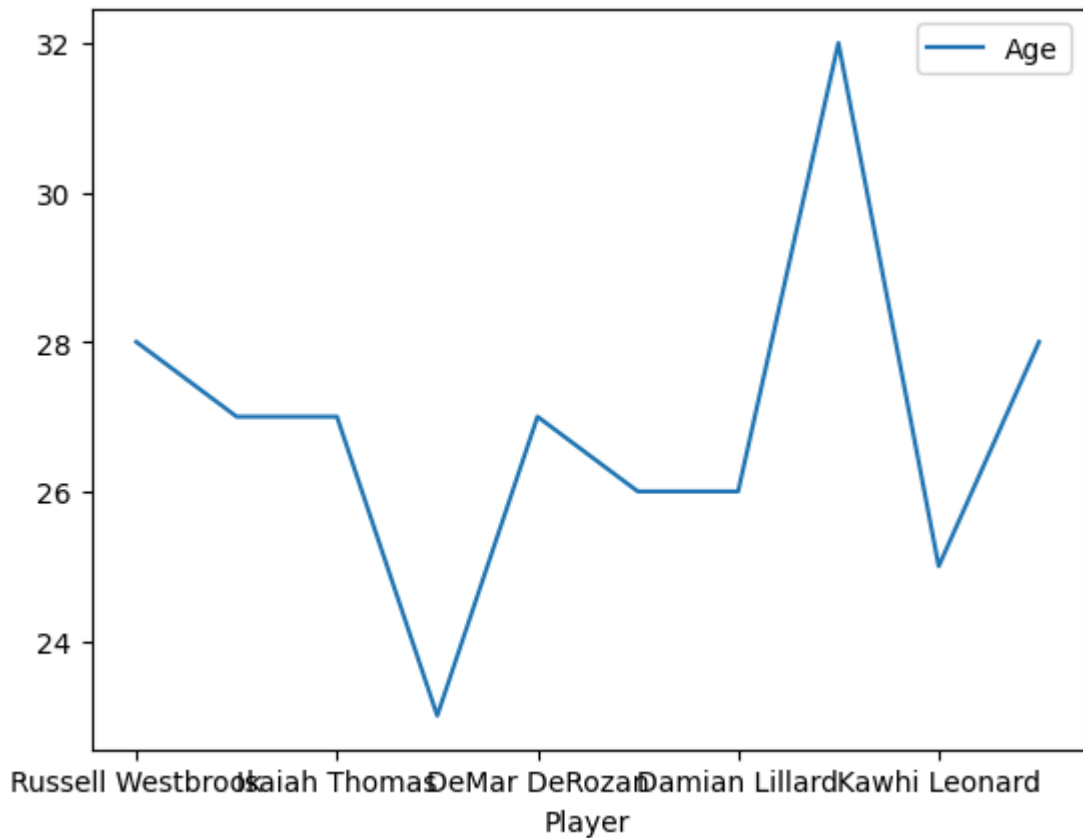


## Line plot

Another useful type of plot is the **line plot**.

```
In [12]: df.head(10).plot(kind='line',  
                           x='Player',  
                           y='Age')
```

```
Out[12]: <Axes: xlabel='Player'>
```

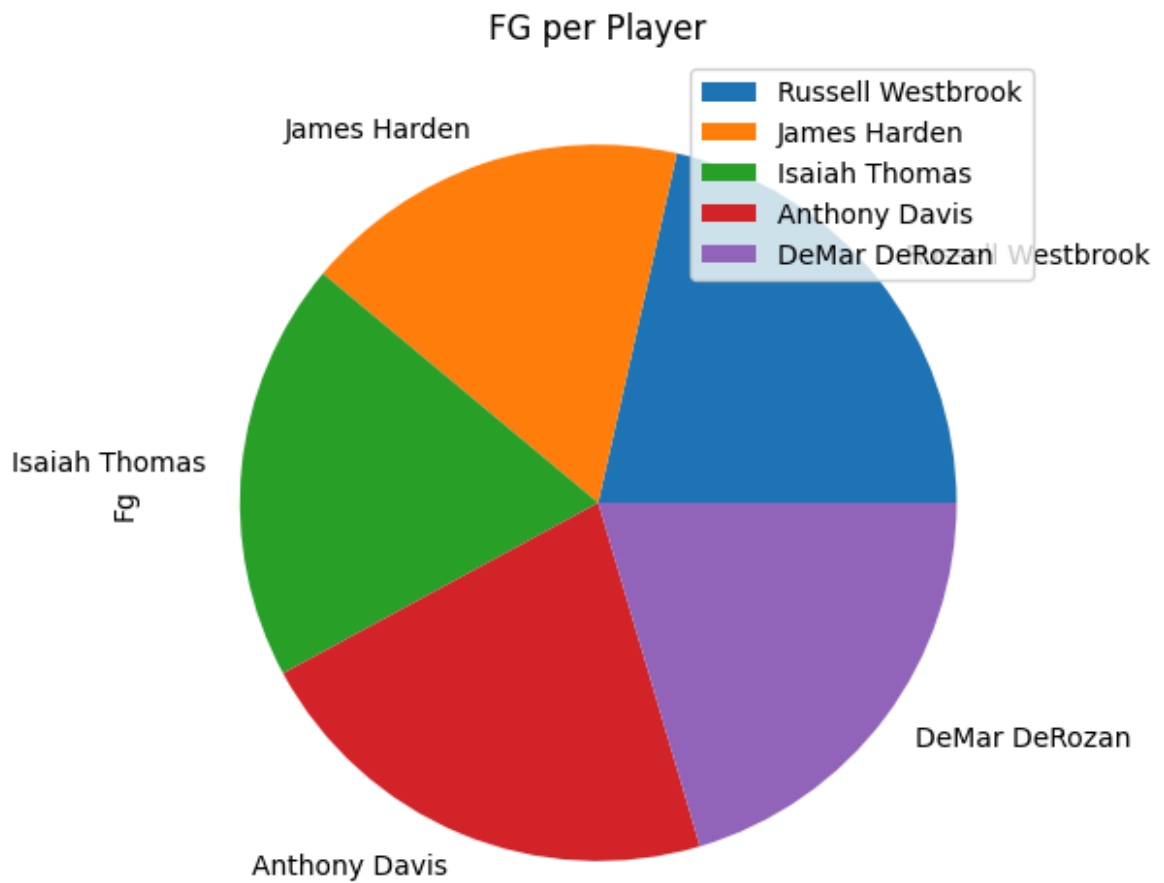


## Pie plot

In order to see the fractional area of different values within a column, we can make a **pie plot**.

```
In [13]: df.head(5)['Fg'].plot(kind='pie',
                                labels=df['Player'],
                                title='FG per Player',
                                legend=True,
                                figsize=(6,6))
```

```
Out[13]: <Axes: title={'center': 'FG per Player'}, ylabel='Fg'>
```



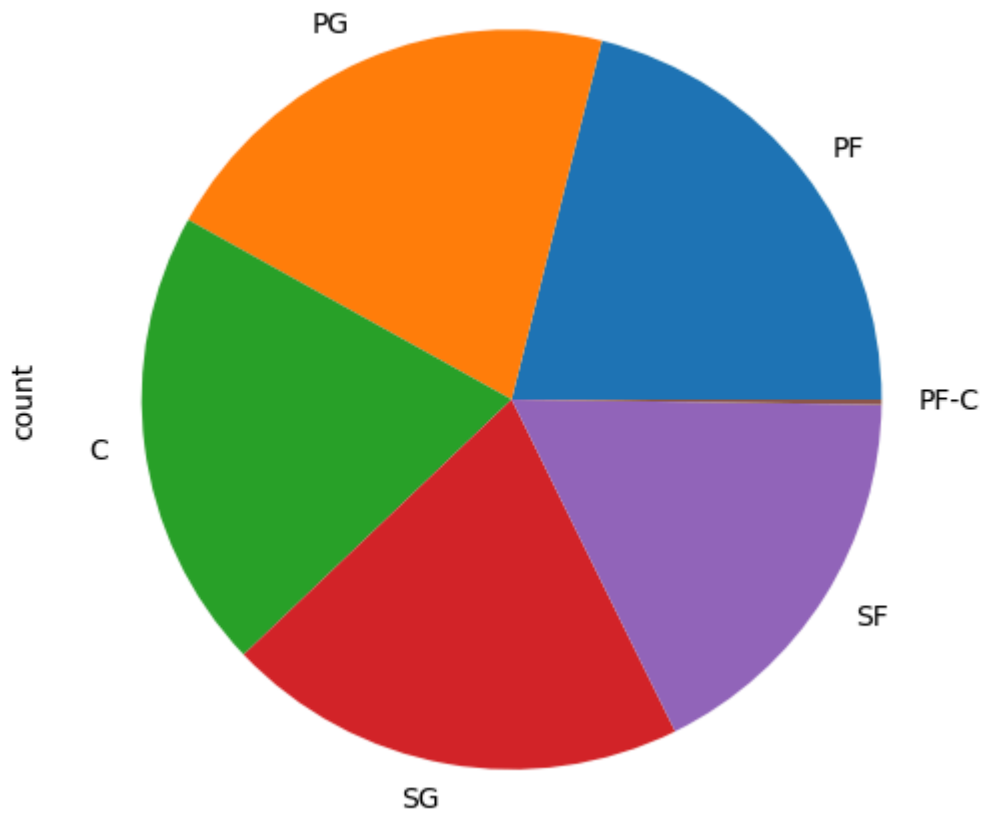
Also is possible with columns without numeric values:

```
In [14]: df['Position'].value_counts()
```

```
Out[14]: Position
PF      94
PG      93
C       90
SG      90
SF      78
PF-C     1
Name: count, dtype: int64
```

```
In [15]: df['Position'].value_counts().plot(kind='pie', figsize=(6,6))
```

```
Out[15]: <Axes: ylabel='count'>
```

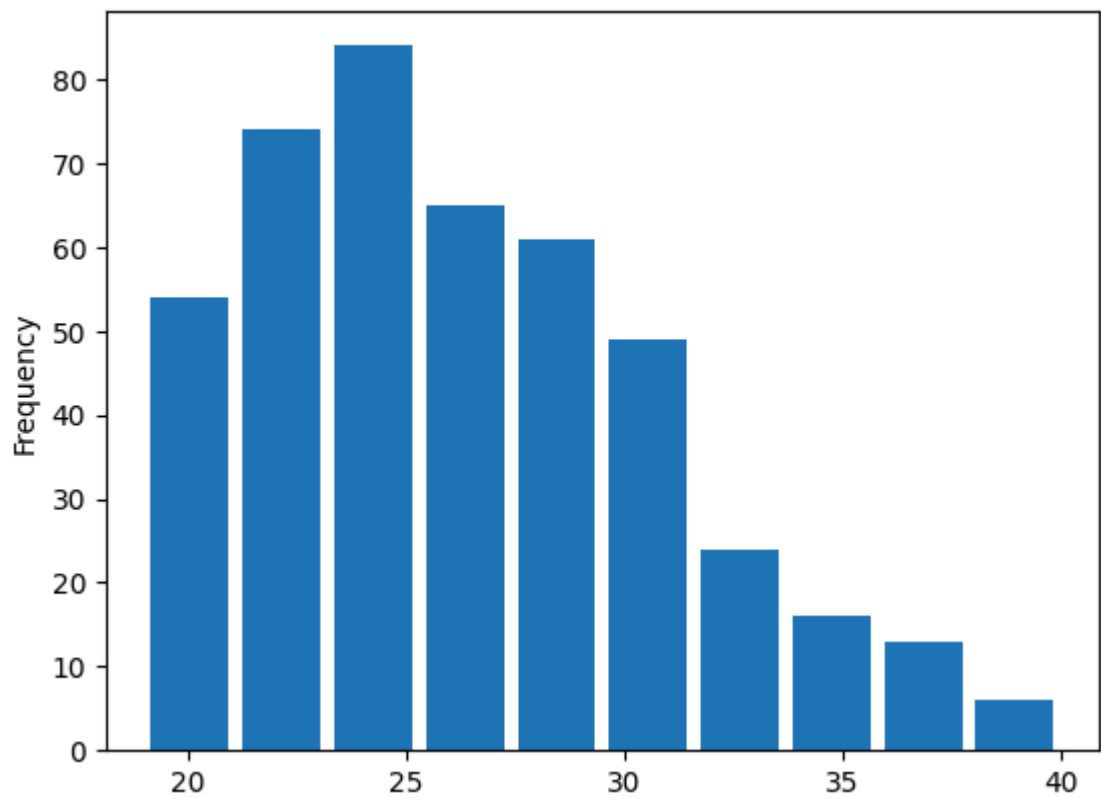


## Histogram

Histograms can also be drawn.

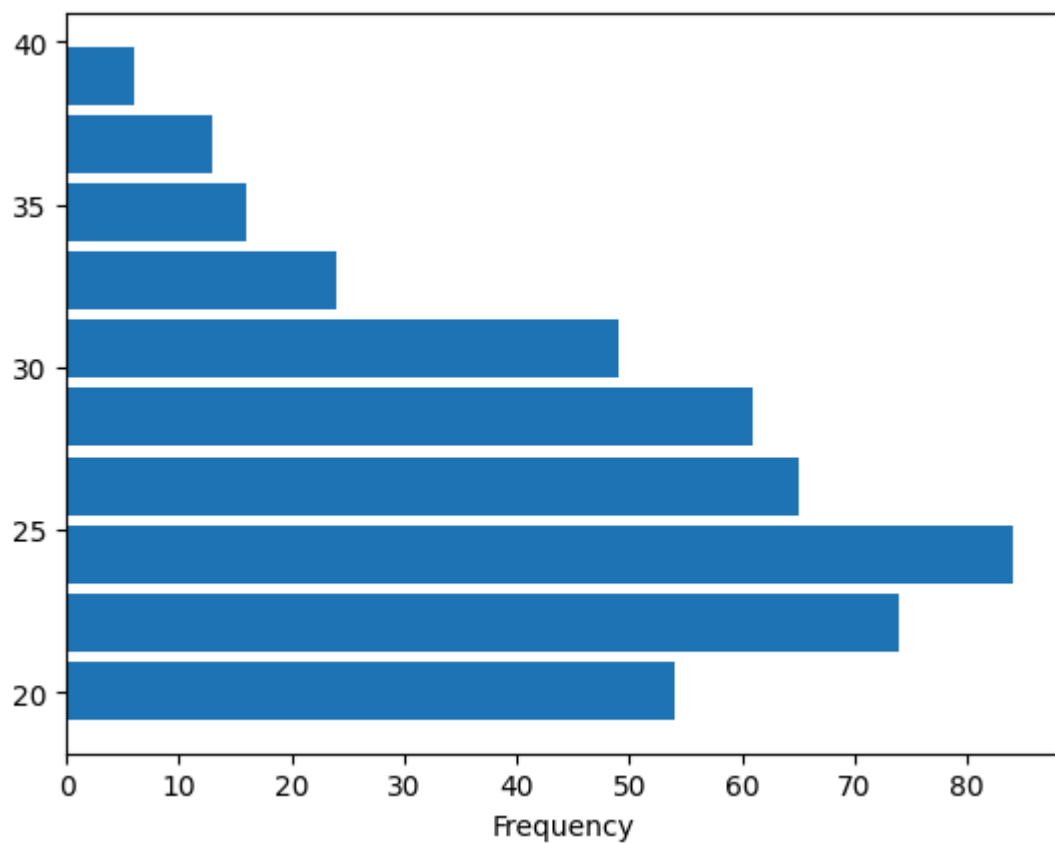
```
In [16]: df['Age'].plot(kind='hist',  
                        rwidth=0.85)
```

```
Out[16]: <Axes: ylabel='Frequency'>
```



```
In [17]: df['Age'].plot(kind='hist',  
                        rwidth=0.85,  
                        orientation='horizontal')
```

Out[17]: <Axes: xlabel='Frequency'>



## Mathematical Examples

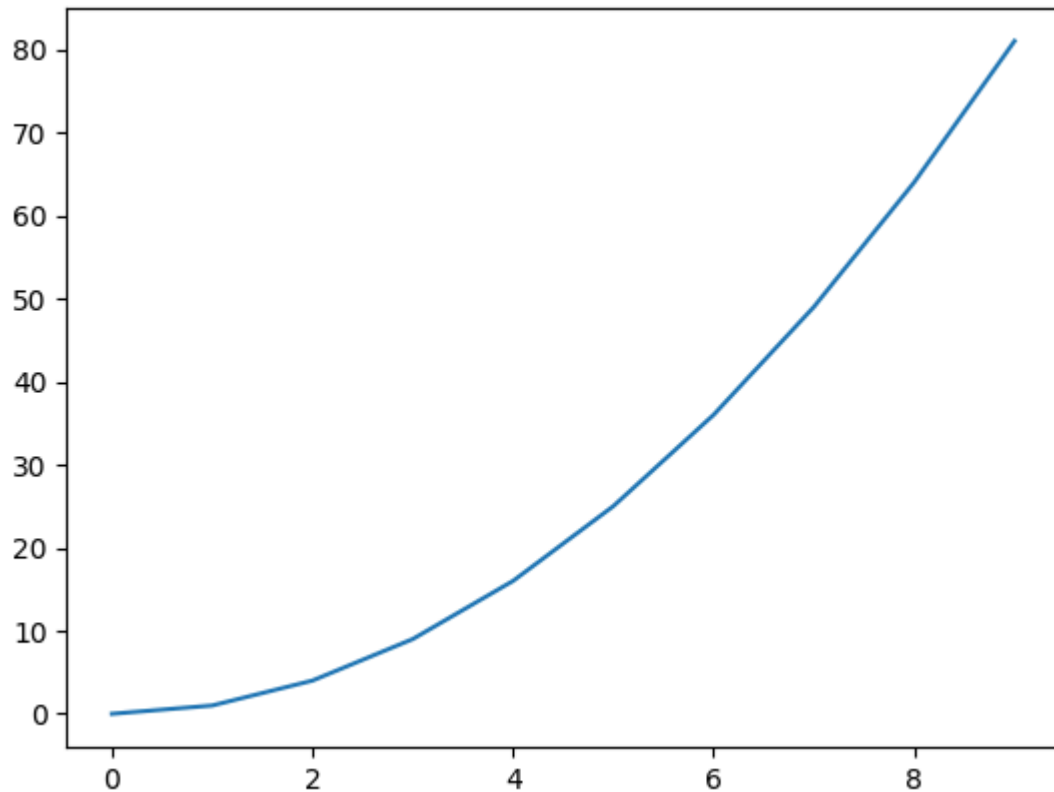
`plt.plot()` accepts many parameters, but the first two ones are the most important ones: the values for the `X` and `Y` axes.

Let's make a plot showing  $f(x) = x^2$ :

```
In [18]: x_values = np.arange(10)
y_values = [value ** 2 for value in x_values]

plt.plot(x_values, y_values)
```

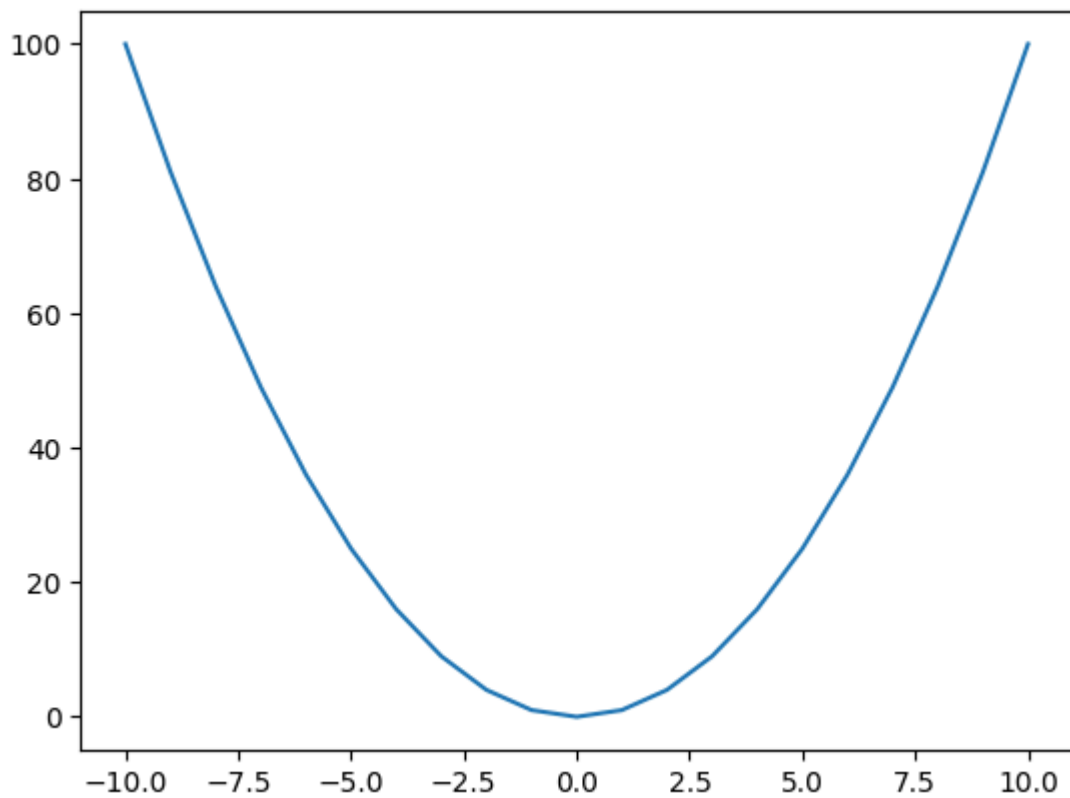
```
Out[18]: [<matplotlib.lines.Line2D at 0x1512be9f260>]
```



```
In [19]: x_values = np.arange(-10, 11)
y_values = [value ** 2 for value in x_values]

plt.plot(x_values, y_values)
```

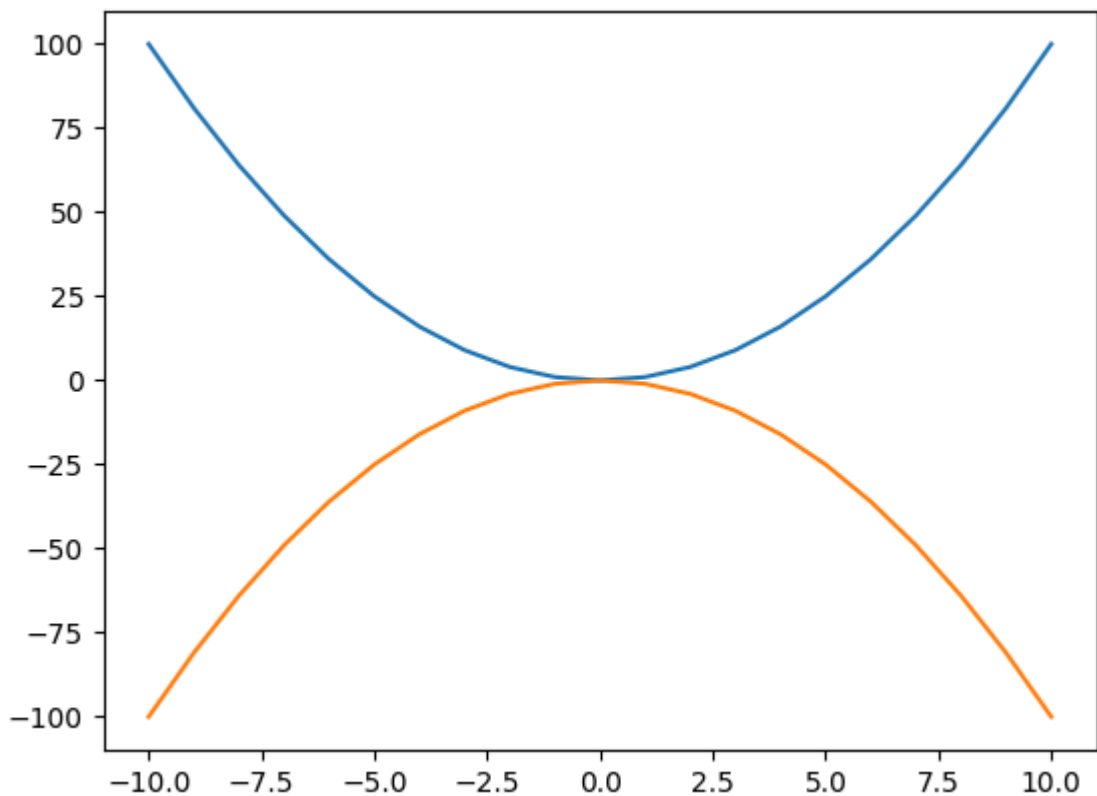
```
Out[19]: [<matplotlib.lines.Line2D at 0x1512cf43320>]
```



We're using `matplotlib`'s global API, which is horrible but it's the most popular one. We'll learn later how to use the *OOP* API which will make our work much easier.

```
In [20]: plt.plot(x_values, x_values ** 2)
plt.plot(x_values, -1 * (x_values ** 2))
```

```
Out[20]: [<matplotlib.lines.Line2D at 0x1512dfc1e20>]
```



Each `plt` function alters the global state. If you want to set settings of your plot you can use the `plt.figure` function. Others like `plt.title` keep altering the global plot:

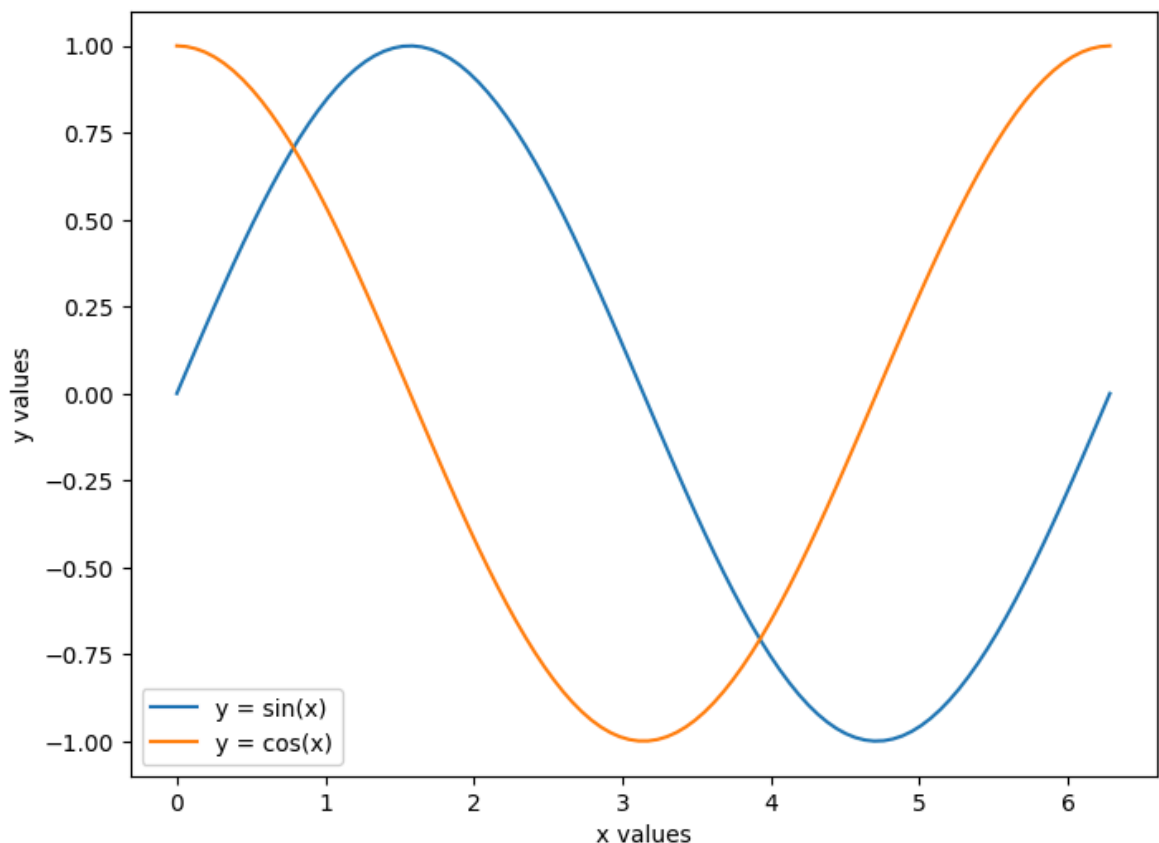
Go a step further and:

- Plot sine and cosine series.
- Add axis names.
- Add a legend.

```
In [21]: plt.figure(figsize=(8,6))

x_values = np.linspace(0, 2 * np.pi, 100)
sin_line, = plt.plot(x_values, np.sin(x_values))
cos_line, = plt.plot(x_values, np.cos(x_values))

plt.xlabel('x values')
plt.ylabel('y values')
plt.legend([sin_line, cos_line], ['y = sin(x)', 'y = cos(x)'])
plt.show()
```



## Replicating Pandas' plot with matplotlib

At its core, pandas is using matplotlib to draw the plots on screen. We can simulate the steps followed with matplotlib to understand a little bit more how it works. This was our original pandas based plot:

Using matplotlib, we need to take a few more steps:

```

In [22]: def backward_difference_heat_eq(nx=101, nt=1001, t0=0, tend=1, C=1.0):
    # Spatial domain
    a, b = 0.0, 1.0
    hx = (b - a) / (nx - 1)
    x = np.linspace(a, b, nx)

    # Time domain
    ht = (tend - t0) / (nt - 1)
    t = np.linspace(t0, tend, nt)

    _lambda = C * ht / hx**2

    # Source function
    def g(x, t):
        return 0.0

    # Initialize solution array
    u = np.zeros((nx, nt))
    u[:, 0] = np.sin(np.pi * x) # Initial condition
    u[0, :] = 0 # Boundary at x=0
    u[-1, :] = 0 # Boundary at x=L

    # Construct tridiagonal matrix A (constant)
    A = np.zeros((nx-2, nx-2))
    for i in range(nx-2):
        if i == 0:
            A[i, i] = 1 + 2*_lambda
            A[i, i+1] = -_lambda
        elif i == nx-3:
            A[i, i-1] = -_lambda
            A[i, i] = 1 + 2*_lambda
        else:
            A[i, i-1] = -_lambda
            A[i, i] = 1 + 2*_lambda
            A[i, i+1] = -_lambda

    # Initial b vector
    gvec = np.array([g(x[i+1], t[1]) for i in range(nx-2)])
    b = u[1:-1, 0] + gvec * ht

    # Time stepping
    for k in range(1, nt):
        # Update for boundary conditions
        b[0] += u[0, k]
        b[-1] += u[-1, k]

        # Solve the linear system
        z = np.linalg.solve(A, b)

        # Update solution
        u[1:-1, k] = z

        # Update source for next time step
        if k+1 < nt:
            gvec = np.array([g(x[i+1], t[k+1]) for i in range(nx-2)])
            b = z + gvec * ht

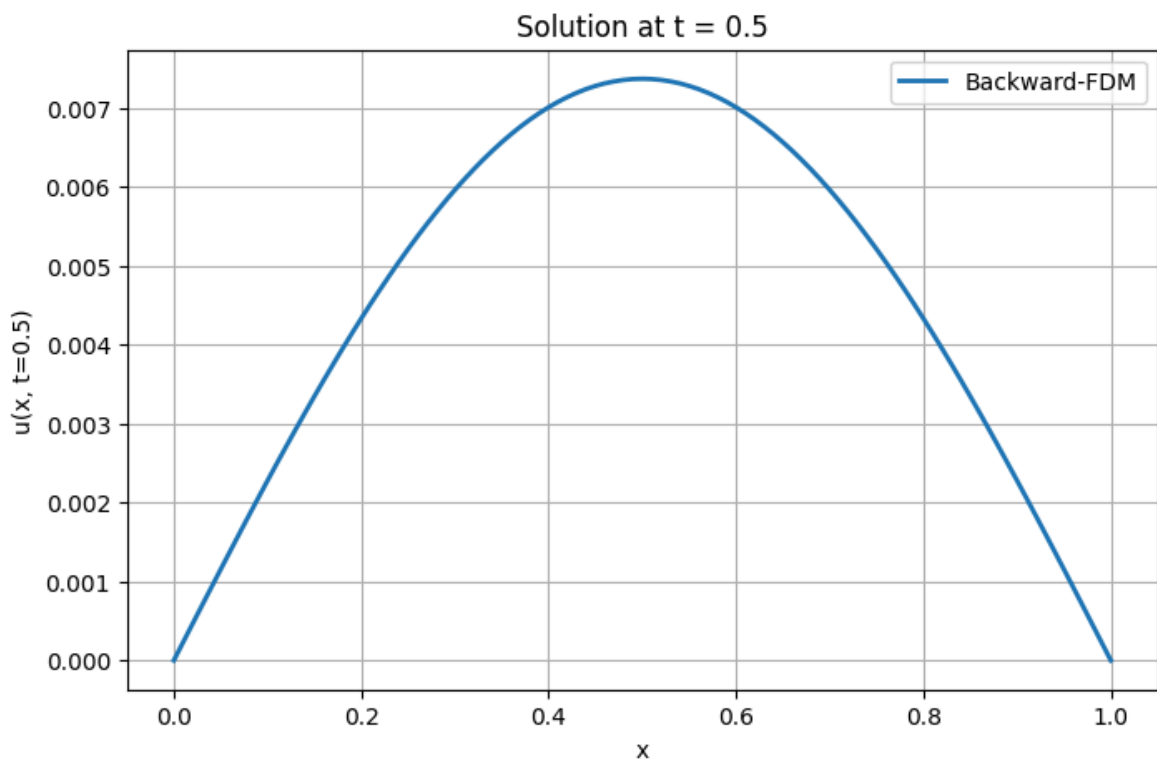
    return x, t, u

```

```
# Run the solver
x, t, u_Bfd = backward_difference_heat_eq()
```

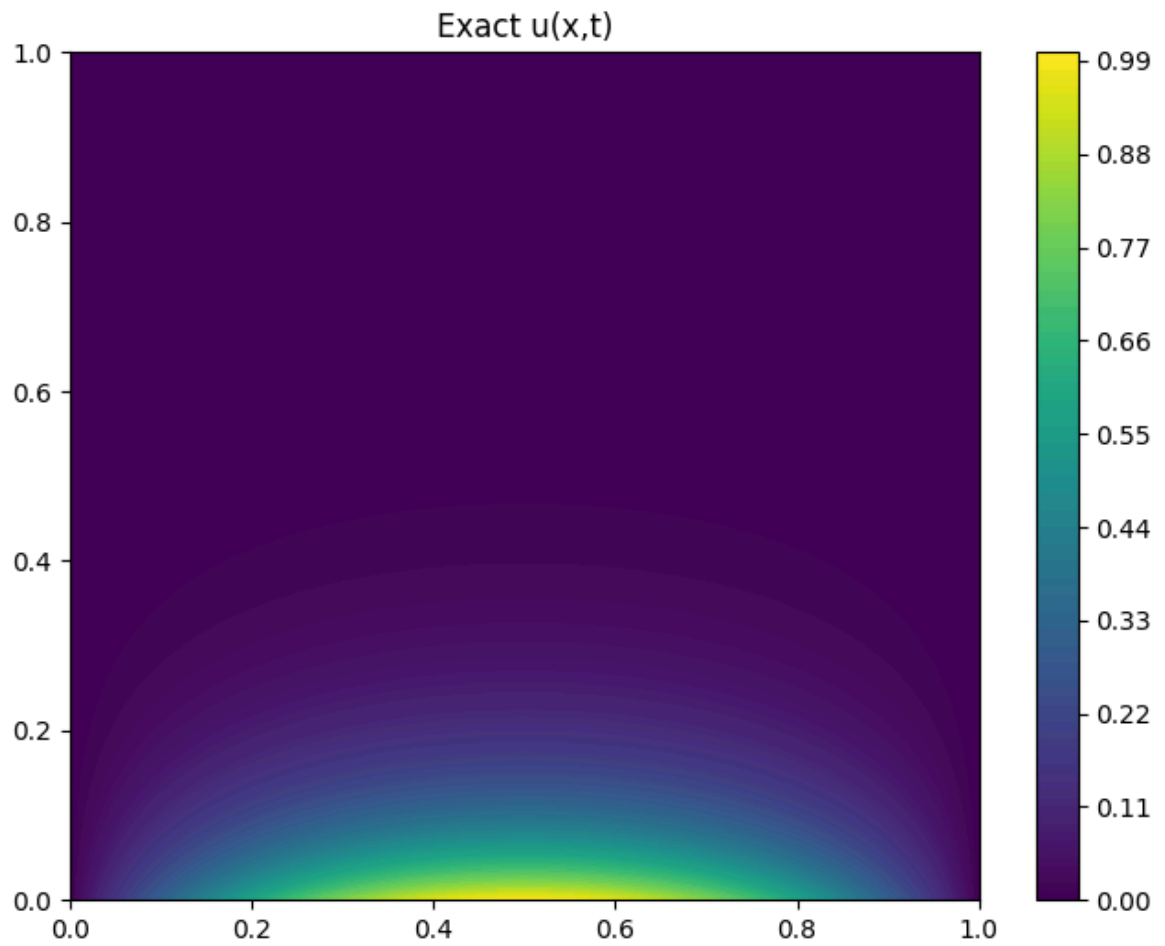
```
In [23]: # -----
# Plot at t = 0.5
# -----

plt.figure(figsize=(8, 5))
plt.plot(x, u_Bfd[:, 500], label='Backward-FDM', linewidth=2)
plt.xlabel("x")
plt.ylabel("u(x, t=0.5)")
plt.title("Solution at t = 0.5")
plt.legend()
plt.grid(True)
plt.show()
```



```
In [24]: x_test = np.linspace(0, 1, 100).reshape(-1, 1)
t_test = np.linspace(0, 1, 100).reshape(-1, 1)
X, T = np.meshgrid(x_test.squeeze(), t_test.squeeze(), indexing="ij")
u_exact = np.sin(np.pi * X) * np.exp(-np.pi**2 * T)

plt.figure(figsize=(8, 6))
plt.title("Exact u(x,t)")
plt.contourf(X, T, u_exact, 100, cmap='viridis')
plt.colorbar()
plt.savefig("exact_u_plot.png", dpi=300, bbox_inches='tight')
plt.show()
```



## Saving the plot

Plots can be saved into images of multiple formats with the `plt.savefig` function: